

Introduction

Les moteurs de recherche occupent aujourd'hui une place centrale dans l'accès à l'information numérique. Ils permettent d'explorer rapidement de très grands ensembles de documents en répondant à des requêtes formulées par les utilisateurs et en classant les résultats selon leur pertinence. Derrière cette apparente simplicité se cache un ensemble complexe d'algorithmes et de structures de données optimisées, notamment pour l'indexation, la recherche et le classement de l'information.

Dans le cadre du projet S4 de l'EPITA, nous avons choisi de concevoir et d'implémenter un moteur de recherche textuel de type « Google », à échelle réduite. L'objectif n'est pas de reproduire un moteur industriel, mais de comprendre et de mettre en œuvre les principes fondamentaux de la recherche d'information, en mettant l'accent sur l'algorithmique et les structures de données plutôt que sur l'interface ou l'utilisation de bases de données existantes.

Ce projet est développé en Rust sous Linux et réalisé par un groupe de quatre étudiants. Il s'inscrit pleinement dans les attentes du module S4, en proposant un logiciel où l'algorithmique joue un rôle central, tout en restant suffisamment ambitieux et extensible.

Objectifs du projet

L'objectif principal du projet est de développer un moteur de recherche capable d'indexer un ensemble de documents textuels et de répondre efficacement à des requêtes formulées par l'utilisateur. Le moteur devra être capable de parcourir un corpus de documents, d'en extraire le contenu textuel, de construire une structure d'index adaptée, puis de retourner une liste de documents classés par pertinence en fonction d'une requête donnée.

Le projet vise également à mettre en pratique plusieurs notions fondamentales de l'informatique, notamment la manipulation de chaînes de caractères, la conception de structures de données efficaces, l'analyse de la complexité algorithmique, ainsi que la gestion de la mémoire et des performances dans un langage bas niveau sécurisé comme Rust.

Enfin, ce projet a pour objectif de produire un logiciel robuste, modulaire et bien documenté, respectant les contraintes de codage imposées par l'EPITA, et pouvant être présenté de manière claire et convaincante lors des soutenances.

État de l'art

Les moteurs de recherche modernes reposent principalement sur le concept d'index inversé. Contrairement à une approche naïve consistant à parcourir l'ensemble des documents à chaque requête, l'index inversé permet d'associer à chaque mot la liste des documents dans lesquels il apparaît. Cette structure rend les recherches beaucoup plus rapides et constitue la base de la majorité des moteurs de recherche actuels.

Avant la construction de l'index, les documents doivent subir un traitement linguistique. Ce traitement inclut généralement la tokenisation, qui consiste à découper le texte en mots, la normalisation des caractères (mise en minuscules, suppression de la ponctuation), ainsi que la suppression des mots très fréquents appelés « stop words ». Dans certains cas, un algorithme de stemming ou de lemmatisation est également utilisé afin de regrouper différentes formes d'un même mot sous une forme commune.

Une fois l'index construit, le moteur de recherche doit être capable de traiter les requêtes des utilisateurs. Les modèles les plus simples reposent sur la recherche booléenne, combinant les mots-clés à l'aide d'opérateurs logiques tels que AND ou OR. Cependant, pour améliorer la pertinence des résultats, des modèles de classement plus avancés sont généralement utilisés.

Parmi ces modèles, le TF-IDF (Term Frequency – Inverse Document Frequency) est l'un des plus classiques. Il attribue un score à chaque document en fonction de la fréquence des mots de la requête dans ce document et de la rareté de ces mots dans l'ensemble du corpus. Ce modèle, bien que relativement simple, est encore largement utilisé et constitue un excellent compromis entre efficacité et facilité d'explication. Des variantes plus avancées comme BM25 améliorent encore la qualité du classement, au prix d'une complexité légèrement supérieure.

Solution retenue

Pour ce projet, nous avons choisi d'implémenter un moteur de recherche reposant sur un index inversé construit à partir d'un corpus de documents textuels. Le langage Rust a été retenu en raison de ses performances, de sa gestion sûre de la mémoire et de sa capacité à produire des logiciels robustes sans sacrifier l'efficacité.

Le moteur fonctionnera entièrement en local et proposera une interface en ligne de commande. Cette approche permet de se concentrer sur les algorithmes de recherche

et de classement, sans introduire de complexité inutile liée à une interface graphique ou à une architecture web.

Le cœur du système sera constitué d'un module d'indexation chargé de construire et de stocker l'index inversé, ainsi que d'un module de requêtage chargé d'interpréter les requêtes utilisateur et de calculer les scores de pertinence des documents. Le modèle de classement principal sera basé sur TF-IDF, avec la possibilité d'ajouter des améliorations comme BM25 si le temps le permet.

Description fonctionnelle détaillée

Le moteur de recherche commencera par une phase d'ingestion du corpus. Cette phase consiste à parcourir un ou plusieurs répertoires contenant des documents textuels, à lire leur contenu et à leur attribuer un identifiant unique. Dans le cas de fichiers HTML, une extraction simplifiée du texte sera effectuée afin de supprimer les balises.

Le texte de chaque document sera ensuite traité par un module de normalisation. Ce module découpera le texte en mots, convertira les caractères en minuscules, supprimera la ponctuation et éliminera les mots trop fréquents. Ce traitement garantit une meilleure cohérence lors de l'indexation et de la recherche.

Une fois le texte normalisé, l'index inversé sera construit. Pour chaque mot rencontré, le moteur enregistrera la liste des documents dans lesquels il apparaît, ainsi que des informations supplémentaires telles que la fréquence du mot dans chaque document. Ces données seront stockées dans des structures de données efficaces, puis sauvegardées sur disque afin de pouvoir être réutilisées sans reconstruire l'index à chaque exécution.

Lorsqu'un utilisateur formulera une requête, celle-ci sera analysée afin d'identifier les mots-clés et les éventuels opérateurs logiques. Le moteur récupérera alors les listes de documents associées à chaque mot et les combinera selon la logique de la requête. Les documents candidats seront ensuite classés à l'aide de l'algorithme TF-IDF, et les résultats les plus pertinents seront affichés à l'utilisateur, accompagnés d'un court extrait du texte.

Répartition du travail au sein du groupe

Le travail est réparti entre les quatre membres du groupe de manière à équilibrer les charges et à garantir une bonne séparation des responsabilités. Chaque membre est responsable d'un ensemble cohérent de fonctionnalités, tout en participant aux décisions de conception globales.

Un premier membre est chargé du cœur de l'indexation et du stockage des données. Il conçoit les structures de données de l'index inversé, implémente les mécanismes de sauvegarde et de chargement sur disque, et veille aux performances globales du moteur.

Un second membre se concentre sur le traitement du texte. Il implémente la tokenisation, la normalisation, la gestion des stop words et, si le temps le permet, un algorithme de stemming. Il est également responsable des tests unitaires associés à ces fonctionnalités.

Un troisième membre est responsable du moteur de requêtes et du classement des résultats. Il implémente l'analyse des requêtes utilisateur, la combinaison des résultats et le calcul des scores de pertinence à l'aide de TF-IDF, ainsi que les éventuelles extensions comme BM25.

Enfin, le quatrième membre s'occupe de l'interface en ligne de commande, de l'intégration globale du projet et de la documentation. Il est responsable du bon fonctionnement de l'application dans son ensemble, de la rédaction du README, ainsi que du site web et des rapports demandés pour les soutenances.

Planning prévisionnel

Le projet débute au mois de janvier avec une phase de conception et de prototypage. Cette phase permet de définir précisément l'architecture du moteur, les formats de données et les interfaces entre les modules. Un premier prototype capable d'indexer un petit corpus et de répondre à des requêtes simples est attendu à la fin de cette période.

Le mois de février est consacré à l'implémentation complète de l'index inversé, du moteur de requêtes et de l'algorithme de classement. À la fin de ce mois, le moteur doit être fonctionnel sur un corpus de taille significative et produire des résultats pertinents.

Le mois de mars est dédié à la stabilisation du projet et à la préparation de la première soutenance. Les fonctionnalités principales doivent être opérationnelles, et une démonstration fiable doit être possible.

Les mois d'avril et de mai permettent d'améliorer le moteur, d'ajouter des fonctionnalités optionnelles et d'optimiser les performances. Cette période est également consacrée à la rédaction finale du rapport, à la mise à jour du site web et à la préparation de la soutenance finale.

Conclusion

Ce projet de moteur de recherche constitue un excellent support pour mettre en œuvre des algorithmes fondamentaux de recherche d'information dans un cadre concret et exigeant. Il permet d'aborder des problématiques réelles de performance, de pertinence et de structuration des données, tout en respectant les contraintes techniques et pédagogiques du projet S4 de l'EPITA.

Grâce à une répartition claire des tâches et à une progression planifiée, ce projet offre un cadre solide pour démontrer les compétences acquises en algorithmique, en programmation système et en travail d'équipe.